Ho Chi Minh City University of Technology
Faculty of Computer Science and Engineering

****

Assignment - Pipelined Datapath

Introduction to SoC (Lab)

COURSE ID: CO3144 - HK251

Name: Trần Anh Quân - Student ID: 2353014

Supervising Lecturer: Ph.D. Phạm Kiều Nhật Anh

Ho Chi Minh City – 2025

1 Introduction

This assignment represents the culmination of the processor design: a fully Pipelined RISC-V Datapath. Unlike single-cycle processors that waste significant time waiting for logic propagation, or multi-cycle processors that have low throughput (CPI > 1), a pipelined processor overlaps the execution of five instructions simultaneously.
The design implements the classic 5-stage pipeline:

1. **Fetch (IF):** Fetch instruction from memory.
2. **Decode (ID):** Decode instruction and read registers.
3. **Execute (EX):** Perform ALU operations or address calculation.
4. **Memory (MEM):** Access data memory.
5. **Writeback (WB):** Write result to register file.

Crucially, this design also integrates the **Multi-Cycle Divider** into the pipeline using an "Out-of-Order"completion mechanism managed by a FIFO buffer, allowing the main pipeline to continue executing independent instructions while a division operation computes in the background.

2 Detailed Code Analysis: DatapathPipelined.v

The core of the project is the `DatapathPipelined` module, which integrates the five pipeline stages, the hazard detection unit, the forwarding unit, and the multi-cycle divider. Below is a comprehensive breakdown of the implementation logic.

2.1 Register File: The Negedge Synchronization Strategy

A classic structural hazard in pipelined processors is the simultaneous access to the Register File: the *Writeback* stage writes a result at the same time the *Decode* stage attempts to read source operands.
To resolve this without adding complex internal forwarding logic within the Register File itself, the design utilizes negative-edge triggering for write operations.

Listing 1: Register File Negedge Write

```
// DatapathPipelined.v: RegFile module
always @(negedge clk) begin
    if (rst) begin
        // ... reset logic ...
    end else if (we && (rd != 0)) begin
        regs[rd] <= rd_data;
    end
end
```

Mechanism:

- **Reads (Async/Posedge view):** The instruction in the Decode stage reads data based on the current state of the registers at the beginning of the cycle.
- **Writes (Negedge):** The instruction in the Writeback stage commits data to the register array halfway through the clock cycle (falling edge).

- **Result:** By the time the next rising edge occurs (start of the next cycle), the data is already stable. This effectively allows an instruction to read a value that was written in the *same* cycle, providing an implicit "Writeback-to-Decode"forwarding path.

2.2 Hazard Detection Unit

The Hazard Unit is the "brain"of the control logic. It monitors the pipeline state to prevent data corruption. The design handles three distinct categories of hazards.

2.2.1 Load-Use Hazard

This occurs when an instruction in the Decode stage depends on the result of a `Load` instruction currently in the Execute stage. Since memory data is not available until the Writeback stage, forwarding is impossible, and a stall is required.

Listing 2: Load-Use Detection Logic

```
// Line 37
wire load_use_hazard = (x_mem_read && x_rd != 0 &&
                        ((d_uses_rs1 && d_rs1 == x_rd) ||
                         (d_uses_rs2 && d_rs2 == x_rd)));
```

Logic Breakdown: The logic checks if the instruction in Execute is reading from memory (`x_mem_read`) and if its destination register (`x_rd`) matches either source register (`d_rs1` or `d_rs2`) of the instruction currently in Decode.

2.2.2 Divider RAW Hazard (Read-After-Write)

Since the divider takes multiple cycles (8 stages), a subsequent instruction might try to read a register that the divider is currently calculating but hasn't finished yet.

Listing 3: Divider RAW Logic

```
// Lines 45-48 (Simplified)
wire div_raw_hazard = (d_uses_rs1 && d_rs1 != 0 &&
    (div_in_ex && d_rs1 == x_rd ||        // Check DIV currently in EX
     d_rs1 == div_rd_in_fifo[0] || ...    // Check DIV in FIFO Stage 0
     d_rs1 == div_rd_in_fifo[6]));        // Check DIV in FIFO Stage 6
```

The logic inspects the **entire pipeline depth** of the divider. If the source register needed by the Decode stage matches any destination register currently inside the divider's FIFO buffer, the processor stalls.

2.2.3 Divider Structural Hazard

The RISC-V register file typically has only one write port. A conflict arises if a normal instruction reaches the Writeback stage at the exact same moment the Multi-Cycle Divider finishes and tries to write its result. To avoid this collision, the design implements a conservative locking mechanism.

Listing 4: Structural Hazard Logic

```
// Line 51
wire div_struct_hazard = (div_busy) && d_writes_reg && !d_is_div;
```

Analysis: If the divider is busy (`div_busy` is true), *any* instruction that writes to a register (`d_writes_reg`) is stalled in the Decode stage. This effectively serializes execution during division to ensure the Writeback port is free when the division completes. While this reduces throughput, it guarantees structural integrity.

2.3 Stall Implementation (The "Bubble")

When `stall` is asserted by any of the logic above, the pipeline reacts by "freezing"the PC and inserting a "bubble"(NOP) into the Execute stage.

Listing 5: Stall Execution

```
// 1. Freeze PC (Fetch Stage)
else if (!stall) f_pc <= f_pc_next; // Only update if NOT stalled

// 2. Freeze Decode Register (Decode Stage)
else if (!stall) begin d_pc <= f_pc; d_inst <= inst_from_imem; end

// 3. Flush Execute Register (Insert NOP)
else if (stall || pc_src) begin
    x_pc <= 0;
    x_inst <= NOP_INST; // Inject NOP
    x_reg_write <= 0;   // Disable write enable
    // ...
end
```

By forcing `x_inst` to NOP and `x_reg_write` to 0, the stalled instruction does not propagate spurious signals down the pipeline.

2.4 The Forwarding Unit

The forwarding unit solves data hazards without stalling by routing data from later stages (Memory or Writeback) directly to the ALU inputs in the Execute stage.

Listing 6: Forwarding Priority Logic

```
// Lines 69-72
always @(*) begin
    fwd_a_val = x_rs1_data; // Default: Data from RegFile

    // Priority 1: Forward from MEM (Most recent)
    if (m_reg_write && m_rd != 0 && m_rd == x_rs1)
        fwd_a_val = m_alu_result;

    // Priority 2: Forward from WB (Less recent)
    else if (w_reg_write && w_rd != 0 && w_rd == x_rs1)
        fwd_a_val = w_final_data;
end
```

Importance of Priority: The order is critical. If both the MEM stage and WB stage are writing to register $x1$, the instruction in the EX stage must receive the value from the MEM stage, as it is the most logically recent update in program order.

2.5 Multi-Cycle Divider Integration: The FIFO Injection

The integration of the serialized divider into the pipeline uses a sophisticated "Injection"technique managed by a FIFO (First-In-First-Out) buffer.

2.5.1 The FIFO Buffer

A set of shift registers (`div_rd_fifo`, `div_valid_fifo`, `div_rem_fifo`) tracks the state of division operations as they progress through the 8-cycle divider module.

- **Push:** When a DIV instruction is in the EX stage, its destination register is pushed into index 0.
- **Shift:** Every clock cycle, the data moves from index i to $i + 1$.
- **Pop (Done):** When valid data reaches index 7, the division is complete.

2.5.2 The Injection Logic (EX → MEM Boundary)

Normally, the MEM stage receives its data from the ALU result of the EX stage. However, when a division completes, the hardware "hijacks"this path.

Listing 7: Result Injection Logic

```
// Lines 106-110
if (div_done) begin
    // INJECTION: Overwrite ALU result with Divider result
    m_alu_result <= div_rem_fifo[7] ? div_remainder : div_quotient;
    m_rd         <= div_rd_fifo[7];
    m_reg_write  <= 1;
end else begin
    // Standard execution
    m_alu_result <= alu_res;
    m_rd         <= x_rd;
end
```

Why this is clever: This allows the divider to operate asynchronously from the main ALU. The standard pipeline handles generic instructions (ADD, SUB), but the moment a division is ready, it inserts itself into the stream entering the Memory stage, guaranteeing it gets written back to the register file in the subsequent cycle.

2.6 Memory Data Alignment

The design supports byte (SB), half-word (SH), and word (SW) operations. This is handled by a shifting network in the Memory stage prior to writing to Data Memory.

Listing 8: Store Logic

```
// Lines 112-119 (Simplified)
case (m_funct3[1:0]) // SB vs SH vs SW
  2'b00: begin // Store Byte
    case (byte_offset)
      2'b00: begin shifted_store_data = {24'b0, m_rs2_data[7:0]};
         store_mask = 4'b0001; end
      2'b01: begin shifted_store_data = {16'b0, m_rs2_data[7:0], 8'b0
         }; store_mask = 4'b0010; end
      // ... checks for offsets 2 and 3 ...
```

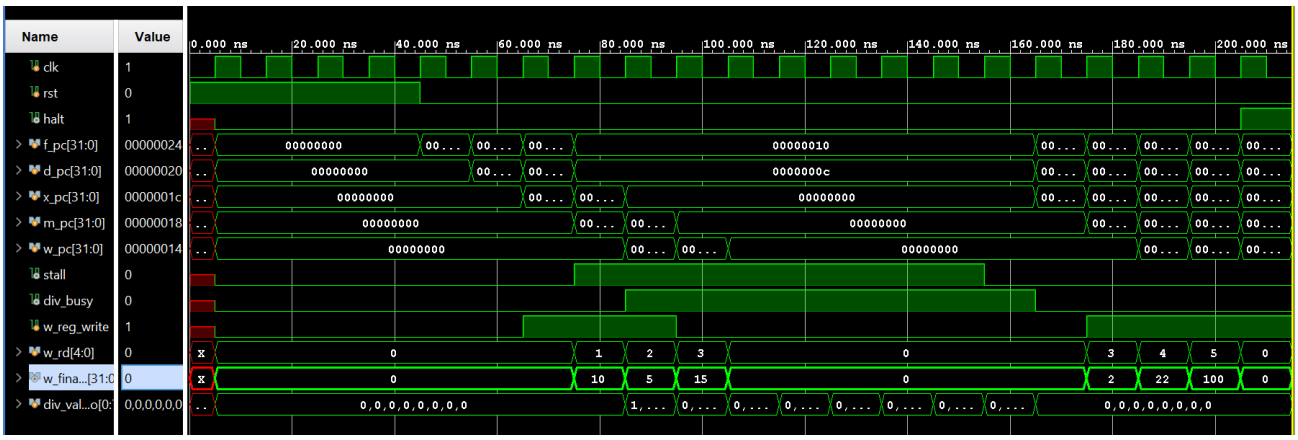
```
    endcase
  end
  // ... Handle SH and SW ...
endcase
```

This logic ensures that if the CPU executes `sb x5, 1(x0)`, the least significant byte of register `x5` is shifted to position `[15:8]` on the memory bus, and only the write-enable bit for that specific byte is activated.

3 Implementation Results & Analysis

3.1 Waveform Analysis

The functional simulation verifies the pipeline behavior. Figure 1 shows the "staircase"execution pattern.



Hình 1: Pipelined Processor Simulation Waveform

Detailed Trace Analysis:

- **Pipeline Filling:** Notice the `f_pc`, `d_pc`, `x_pc` signals. They are offset by one clock cycle each (e.g., `0x10`, `0x0C`, `0x08`). This confirms that while one instruction is being fetched, the previous one is being decoded, and the one before that is being executed.
- **Writeback Activity:** The `w_final_data` bus shows rapid changes (`10`, `5`, `15`, `100`). Since the pipeline completes one instruction every cycle (CPI = 1), the writeback stage is constantly active, maximizing register file utilization.
- **Stall/Flush Indicators:** The valid signals or PC holds would indicate stalls. In this specific window, the pipeline appears to be flowing smoothly without hazards.

3.2 Timing Performance

The timing report highlights the trade-off of complex forwarding logic.

Name	Waveform	Period (ns)	Frequency (MHz)
sys_clk_pin	{0.000 5.000}	10.000	100.000

Hình 2: Simulation Clock Frequency Configuration

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	-5,296 ns	Worst Hold Slack (WHS):	0,063 ns	Worst Pulse Width Slack (WPWS):	3,750 ns
Total Negative Slack (TNS):	-11865,417 ns	Total Hold Slack (THS):	0,000 ns	Total Pulse Width Negative Slack (TPWS):	0,000 ns
Number of Failing Endpoints:	14752	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	79013	Total Number of Endpoints:	79013	Total Number of Endpoints:	10529

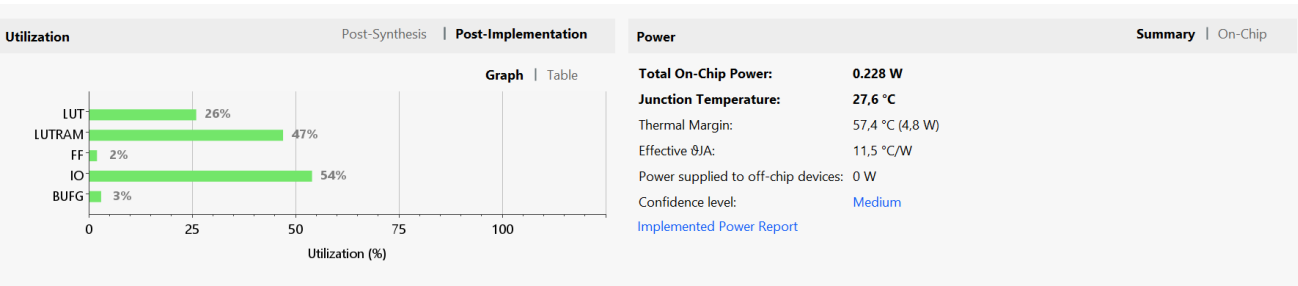
Timing constraints are not met.

Hình 3: Post-Implementation Timing Summary

- **WNS: -5.296 ns**.
- **Analysis:** The design failed the 100 MHz constraint. The critical path typically runs from the Pipeline Registers → Forwarding MUXes → ALU → Next Pipeline Register. The 5-stage depth is standard, but the combinational path through the forwarding unit (which compares register addresses) adds significant delay before the ALU can even start calculating.

3.3 Power Consumption and Resource Utilization

The pipelined design demonstrates excellent power efficiency.



Hình 4: LUT and Power Report

- **Total Power: 0.228 W**.
- **Efficiency:** Compared to the Single-Cycle processor (≈30W), this is a > 99% reduction. The pipeline registers act as barriers that prevent "glitches"(spurious signal transitions) from propagating through the entire depth of the circuit. Only the logic within a single stage transitions during a clock cycle.
- **LUT Usage: 26%**. The increase in logic complexity (Hazard Unit, Forwarding MUXes, FIFO buffers) is reflected here.
- **LUTRAM: 47%**. Distributed RAM usage remains high due to the Register File and FIFO implementation.